Explicit credentials alert (Event 4648): from the alert to the answer

Written for someone starting from nothing. Examples marked MADE-UP use invented names (bob.smith, WS-0142, adm_bob) so you can see the shape of a log or a result. Examples marked LAB come from your sysmon.json. None of the Splunk queries have been run on your Splunk, so check field names against one real event first.

1. The story in one minute

Every Windows computer keeps a logbook called the Event Log. Splunk collects the logbooks from all the computers so you can search them in one place. Your alert comes from one line in that logbook, event number 4648. That line means: someone showed an ID card that was not their own.

Picture a school. A teacher holds up the principal's card to get into a room. That is not automatically bad: the principal may have asked. It is bad when a stranger does it, or when a teacher uses the principal's card to enter a room they never go into. 'Explicit credentials' just means a username and password were handed over directly, instead of using the account that was already signed in.

So you are answering four questions. Who held up the card? Whose card was it? Which program held it up? Which room were they trying to enter?

One important thing: 4648 says someone TRIED. It does not say it worked. Whether it worked shows up as separate events: 4624 means a sign-in worked and 4625 means it failed.

2. What you have, and where the rest is

Your alert gave you two labels, the username and the host name. Think of them as the label on a folder. The folder is the full event, and it holds the answers to all four questions. You open the folder with this search:

```
index=* sourcetype=*WinEventLog:Security EventCode=4648 host="WS-0142" "bob.smith" earliest=-24h
| table _time host _raw
```

Part of the query	What it means, like you are a kid
index=*	Look on every shelf of the library. Later swap the * for the real shelf name, because looking everywhere is slow.
sourcetype=*WinEventLog:Security	Only the Windows Security logbook. The star at the start makes it match both ways Splunk can label it.
EventCode=4648	Only the pages of type 4648.
host="WS-0142"	Only that computer. Put the host name from your alert here.
"bob.smith"	The username in quotes. Splunk finds that word anywhere in the event.
earliest=-24h	Only the last 24 hours. -30d means 30 days.
| (the pipe)	Means: now take that result and do the next thing, like an assembly line.
table _time host _raw	Show only these columns: the time, the computer, and _raw, which is the whole original text of the event.

Click the small arrow next to the event to open it. This is what you will see.

3. The event itself, line by line (MADE-UP)

```
A logon was attempted using explicit credentials.

Subject:
    Security ID:           S-1-5-21-1111111111-2222222222-3333333333-1104
    Account Name:          bob.smith
    Account Domain:        CORP
    Logon ID:              0x3E7A1
    Logon GUID:            {00000000-0000-0000-0000-000000000000}

Account Whose Credentials Were Used:
    Account Name:          adm_bob
    Account Domain:        CORP
    Logon GUID:            {00000000-0000-0000-0000-000000000000}

Target Server:
    Target Server Name:    localhost
    Additional Information: localhost

Process Information:
    Process ID:            0x1A4C
    Process Name:          C:\Windows\System32\consent.exe

Network Information:
    Network Address:       -
    Port:                  -

This event is generated when a process attempts to log on an account by explicitly
specifying that account's credentials. This most commonly occurs in batch-type
configurations such as scheduled tasks, or when using the RUNAS command.
```

Line	What it means
A logon was attempted using explicit credentials.	The headline. Somebody tried to get in by handing over a username and password directly. Tried, not necessarily succeeded.
Subject:	The heading for 'who is doing this'. This is the person who was already signed in.
Security ID	The barcode on the person's card (called a SID). Names can change or be shared, a barcode cannot. If two people have the same name, the barcode tells them apart.
Account Name: bob.smith	The signed-in person. Does it match the user in your alert?
Account Domain: CORP	Which team or school the card belongs to. If the card only works on one computer, this shows the computer's own name instead.
Logon ID: 0x3E7A1	The ticket number of bob's current visit to the computer. Every visit gets one. The 0x means the number is written in hex. You use it in step E to find where this visit started.
Logon GUID	A very long unique tag for the visit, used to match records on the domain controller. All zeros is normal when the sign-in did not use Kerberos.
Account Whose Credentials Were Used:	The heading for 'whose ID card was shown'. This is the most important block.
Account Name: adm_bob	The card that was shown. If it is different from the Subject, someone else's or an admin account's credentials were supplied. Is it an admin card?
Account Domain / Logon GUID (second block)	Same ideas as above, but for the card that was shown.
Target Server:	The heading for 'which room were they trying to enter'.
Target Server Name: localhost	localhost means this same computer. A different server name means they were trying to reach another machine.
Additional Information	An extra label about the room. For Remote Desktop it often looks like TERMSRV/servername.
Process Information:	The heading for 'which program held up the card'.
Process ID: 0x1A4C	The running program's seat number (hex, 0x1A4C is 6732). Windows reuses these numbers, so it is a clue for a short time, not a permanent name.
Process Name: ...consent.exe	The program's full address. The most important line. consent.exe is the UAC box that asks for a grown-up's password.
Network Information:	The heading for the street address of the room.
Network Address / Port	The IP address and door number of the target. A dash means the same computer. Port 3389 is Remote Desktop, 445 is file sharing, 5985 and 135 are remote management.
This event is generated when...	Windows prints this explanation in every 4648 event. It says the usual reasons are scheduled tasks and runas, which is why those are the routine cases.

Reading this one as a story: bob.smith, a normal user, was working at his own computer (the target is localhost and there is no network address). A program called consent.exe, the UAC box, asked for admin rights, and someone typed in the credentials of adm_bob. The most likely explanation is that IT helped bob install or change something. The question to ask bob is: did IT help you around 09:14? If bob says no, then either bob knows an admin password, which is a problem, or something typed it for him.

4. Five example events and what to think (MADE-UP)

Program	Target	Card used	What it probably is	What you do
consent.exe	localhost	adm_bob	IT or the user raising their rights with an admin password	Confirm with the user. Check what ran next (step C).
taskhostw.exe	localhost	svc_sched	A scheduled task running under a stored account	Check it is a known task and it appears in the last 30 days (step B).
mstsc.exe	TERMSRV/SQL01, port 3389	adm_bob	Remote Desktop to a server using a typed admin password	Does bob manage SQL01? First time? Check step D.
net.exe	\\DC01\C$, port 445	adm_domain	net use to a domain controller's hidden drive with a typed admin password	Red flag unless IT. Read the command line in Sysmon. Check DC01.
...\Downloads\invoice.pdf.exe	DC01, port 445	adm_domain	A program from the Downloads folder using an admin card to reach a domain controller	Treat as an incident. Isolate the machine, run D1 and E3 from the pack.

5. The Splunk steps, with example results

Words you will see in every query

Word	What it means
eval x=...	Make a new column called x.
if(test, a, b)	If the test is true use a, otherwise b.
case(test1, a, test2, b)	A sorting hat. It checks the tests in order and uses the first that fits.
coalesce(a, b)	Use a, and if a is empty use b. Different Splunk setups name fields differently, so this makes one query work for both.
lower(x)	Turn into small letters, so BOB and bob count as the same.
mvindex(list, 0)	Pick one item from a list. Counting starts at 0, so 0 is first, 1 is second, -1 is last.
split(text, "\\")	Cut text into pieces at every backslash. Splunk needs two backslashes to mean one real one.
match(text, pattern)	Does the text fit the pattern? Yes or no.
replace(text, pattern, new)	Find the pattern and swap it for something else.
stats ... by x	A tally chart with one row per x.
count	How many.
dc(x)	Distinct count. How many DIFFERENT x.
values(x)	List every different value of x that was seen.
earliest(_time), latest(_time)	The first time and the last time.
bin _time span=10m	Group events into 10 minute buckets.
convert ctime(x)	Turn a computer time number into a readable date.
sort - count / sort 0 _time	Biggest first / oldest first. The 0 means do not cut the list short.
where	Keep only the rows where this is true.
regex	Keep only text that fits a pattern.
isnotnull(x)	True if x has a value.
_raw	The whole original event text.

Pattern symbol (regex)	Meaning
(?i)	Ignore capital letters.
\.	A real dot. A plain dot would mean any character.
$ and ^	The very end and the very start of the text.
a|b	Either a or b.
.*	Anything, any length.
\s+ and \S+	One or more spaces, and one or more non-space characters.
\b	A word edge, so net does not match inside internet.
[^\s]	Any character that is not a slash and not a space.
?	The thing just before it is optional.
* in Splunk searches	A wildcard: anything can go here.

Step A: tidy the alert into a readable row

```
index=* sourcetype=*WinEventLog:Security EventCode=4648 host="WS-0142" "bob.smith" earliest=-24h
| eval from_acct=lower(coalesce(SubjectUserName, mvindex(Account_Name,0))), used_acct=lower(coalesce(TargetUserName,
      mvindex(Account_Name,1)))
| eval program=coalesce(ProcessName, Process_Name), target=coalesce(TargetServerName, Target_Server_Name),
      ip=coalesce(IpAddress, Network_Address)
| sort 0 _time
| table _time host from_acct used_acct program target ip
```

Line	What it does
First eval	Makes two columns. from_acct is the person who was signed in (the Subject). used_acct is the card that was shown. Some setups store those as two separate fields, others store Account_Name as a list of two, where item 0 is the person and item 1 is the card. coalesce handles both. lower makes the names small letters.
Second eval	Same trick for the program, the room (target) and the address (ip).
sort 0 _time	Oldest first.
table	Show just those columns.

Example result (MADE-UP):

_time	host	from_acct	used_acct	program	target	ip
2026-10-28 09:14:03	WS-0142	bob.smith	adm_bob	C:\Windows\System32\consent.exe	localhost	-

This is the folder opened: bob, using adm_bob's card, through the UAC box, on his own computer.

Step B: is this normal for this person?

```
index=* sourcetype=*WinEventLog:Security EventCode=4648 "bob.smith" earliest=-30d
| eval program=lower(mvindex(split(coalesce(ProcessName,Process_Name),"\\"),-1)),
      target=coalesce(TargetServerName,Target_Server_Name)
| stats count, earliest(_time) as first, latest(_time) as last by program, target
| convert ctime(first) ctime(last)
| sort - count
```

Line	What it does
Search line	Same search but for 30 days, and without the host so you see this user everywhere.
eval	ProcessName is a full path such as C:\Windows\System32\consent.exe. split cuts it at every backslash, mvindex(...,-1) takes the last piece (just consent.exe), and lower makes it small letters.
stats	The tally chart: for each program and target pair, how many times, and the first and last time.
convert, sort	Readable dates, most common first.

Example result (MADE-UP):

program	target	count	first	last
taskhostw.exe	localhost	212	2026-09-28 06:00:02	2026-10-28 06:00:01
consent.exe	localhost	1	2026-10-28 09:14:03	2026-10-28 09:14:03

Reading it: taskhostw.exe has run 212 times in a month, so that is a routine scheduled task. consent.exe happened once, today, so it is new for bob. That does not make it bad. It means ask.

Step C: what happened around the alert

```
index=* host="WS-0142" earliest="10/28/2026:09:00:00" latest="10/28/2026:09:30:00"
  ((sourcetype=*WinEventLog:Security (EventCode=4624 OR EventCode=4625 OR EventCode=4648 OR EventCode=4672 OR
        EventCode=4688 OR EventCode=4698 OR EventCode=4720 OR EventCode=4732))
   OR (sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" (EventCode=1 OR EventCode=3)))
| sort 0 _time
| table _time sourcetype EventCode Logon_Type Account_Name Source_Network_Address New_Process_Name Image CommandLine
      DestinationIp
```

Line	What it does
First line	Everything from that computer between two moments. Use the alert time minus 15 minutes and plus 15 minutes. Dates are month/day/year.
Security events	4624 a sign-in worked. 4625 a sign-in failed. 4648 explicit credentials. 4672 a sign-in was given admin-style powers. 4688 a program started. 4698 a scheduled task was created. 4720 a user was created. 4732 someone was added to a local group.
Sysmon events	Sysmon is a free Microsoft add-on that writes a very detailed diary of what programs do. EventCode 1 is a program started, 3 is a network connection.
table	Show the useful columns.

Example result (MADE-UP):

_time	Event	What it is
09:14:03	4648	Explicit credentials (your alert)
09:14:03	4624	A sign-in for adm_bob worked
09:14:03	4672	Special privileges assigned to adm_bob
09:14:05	Sysmon 1	mmc.exe started: "C:\Windows\system32\mmc.exe" "C:\Windows\system32\dsa.msc"
09:16:40	Sysmon 3	mmc.exe connected to DC01 on port 389

Reading it: right after the card was shown, an admin console called dsa.msc (Active Directory Users and Computers) opened and talked to the domain controller on port 389. That fits an IT task, but it also shows that bob's machine was used to make changes in AD, so confirm with IT and check the changes on DC01 (Part 6, layer 3).

Step D: where else did that card go?

```
index=* sourcetype=*WinEventLog:Security (EventCode=4624 OR EventCode=4648 OR EventCode=4625) "adm_bob" earliest=-24h
| stats count, dc(host) as hosts, values(host) as host_list, values(Logon_Type) as logon_types, earliest(_time) as
      first, latest(_time) as last by EventCode
| convert ctime(first) ctime(last)
```

Line	What it does
Search line	Every sign-in, explicit-credentials event and failed sign-in that mentions the card that was shown (adm_bob), for 24 hours.
stats	One row per event type: how many, how many DIFFERENT computers, which ones, which kinds of sign-in, and when.

Example result (MADE-UP):

EventCode	count	hosts	host_list	logon_types
4624	3	1	WS-0142	2
4648	1	1	WS-0142	

Reading it: the card stayed on one computer. That is calm. The worrying version would show hosts=9 inside 20 minutes, meaning the card is being carried around the building.

Step E: how did bob's visit start?

```
index=* host="WS-0142" sourcetype=*WinEventLog:Security EventCode=4624 "0x3E7A1"
| table _time Logon_Type Account_Name Source_Network_Address Workstation_Name
```

Take the Logon ID from the event (0x3E7A1) and find the sign-in (4624) that started that visit. Example result (MADE-UP):

_time	Logon_Type	Account_Name	Source_Network_Address	Workstation_Name
2026-10-28 08:02:11	2	bob.smith	-	WS-0142

Logon_Type 2 means bob sat at the keyboard. If it said 10 with an outside IP, he came in by Remote Desktop from somewhere else. Logon IDs are only unique on one machine since its last restart, so always search them on that host.

Logon_Type	Meaning
2	Sitting at the keyboard
3	Over the network, like opening a shared folder
4	A scheduled job
5	A service
7	Unlocking the screen
9	runas with network-only credentials
10	Remote Desktop
11	Cached sign-in, laptop away from the domain

6. The programs you asked about

mstsc.exe: the remote control for other people's computers

mstsc.exe is the Remote Desktop client. It opens a window that shows another computer's screen. In a 4648 event it means someone connected to another machine using a username and password they typed or had saved. The room is the Target Server, often written TERMSRV/servername, with port 3389. IT staff do this every day. It is suspicious if the person has no reason to connect anywhere, the destination is a domain controller or a server they never touch, they hit many servers in a short time, it happens at night, or an admin account is used from an ordinary desk PC instead of the jump box. Attackers use it to move from machine to machine. Windows remembers which servers were connected to under the registry path HKCU\Software\Microsoft\Terminal Server Client\Servers, and cmdkey /add:TERMSRV/... pre-loads a saved password.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 Image="*\\mstsc.exe"
      DestinationPort=3389 earliest=-30d
| stats count, dc(DestinationIp) as servers, values(DestinationIp) as dest_ips, earliest(_time) as first,
      latest(_time) as last by Computer, User
| convert ctime(first) ctime(last)
```

Sysmon EventCode 3 is a network connection. This finds connections made by mstsc.exe to port 3389 in 30 days, and tallies them per computer and user. A server that is new to the list is what to look at. Example result (MADE-UP):

Computer	User	count	servers	dest_ips
WS-0142	CORP\bob.smith	3	2	10.1.4.20, 10.1.4.21

consent.exe: the box that asks for a grown-up's password

consent.exe is the UAC prompt, the 'Do you want to allow this app to make changes to your device?' window. It runs as SYSTEM, the most powerful account. A 4648 event with consent.exe usually means a normal user was asked for admin rights and an administrator typed their name and password into the box. Helpdesk staff do this on users' machines all the time. It is suspicious if the person is not IT, the admin was not actually there, or something unusual ran right afterwards. Attackers can bypass UAC, in

which case consent.exe never appears, so not seeing it proves nothing.

```
index=* host="WS-0142" sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
      IntegrityLevel=High earliest="10/28/2026:09:10:00" latest="10/28/2026:09:20:00"
| table _time User ParentImage Image CommandLine
```

Programs that started with admin power in a short window around the alert. IntegrityLevel is the power level: Medium is normal, High is admin, System is the highest. ParentImage is the program that started it, Image is the program itself, CommandLine is the exact command typed. Example result (MADE-UP):

_time	User	ParentImage	Image	CommandLine
09:14:05	CORP\adm_bob	C:\Windows\explorer.exe	C:\Windows\System32\mmc.exe	"mmc.exe" "C:\Windows\system32\dsa.msc"

net.exe and net1.exe: the Swiss army knife

net.exe is built into every Windows computer. net1.exe is the helper it starts behind the scenes, which is why every command shows up twice. It is an external program, so Sysmon sees it even when it is typed into an open command window. These are the commands that matter:

Command	What it does	Why you care
net user bob pass /add	Creates an account (add /domain for an AD account)	Persistence, like jumpadmin in your lab
net user bob newpass	Resets the password	Account takeover
net user bob /active:no or /delete	Disables or deletes an account	Tampering or cleanup
net localgroup administrators bob /add	Makes bob a local admin	Privilege escalation
net group "Domain Admins" bob /add /domain	Adds bob to a domain group	The most serious one
net accounts /minpwlen:... /lockoutthreshold:...	Changes the password and lockout rules	Weakening defences
net use \\server\c$ /user:corp\admin password	Connects to another machine with a typed password	Spreading, and it creates a 4648
net stop <service>	Stops a service	Often used to turn off antivirus
net view / share / session / time	Lists machines, shares and sessions	Looking around

netdom.exe is the domain-management tool from the admin toolkit. It joins and removes computers from the domain, resets a computer's machine password (netdom resetpwd), and builds or queries trusts. A normal user has no reason to run it.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
(Image="*\\net.exe" OR Image="*\\net1.exe" OR Image="*\\netdom.exe")
| eval action=case(
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+user\s+\S+\s+\S+.*/add"), "create user",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+user\s+\S+.*/delete"), "delete user",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+user\s+\S+.*/active:no"), "disable account",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+user\s+[^/\s]\S*\s+[^/\s]\S*"), "possible password reset",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+localgroup\s+.*/add"), "add to local group",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+group\s+.*/add"), "add to domain group",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+use\s+.*/user:"), "net use with explicit credentials",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+accounts\s+.*/(minpwlen|maxpwage|minpwage|uniquepw|lockoutthreshold|force
          logoff)"), "change password or lockout policy",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+stop\b"), "stop service",
    match(CommandLine,"(?i)\bnetdom(\.exe)?\s+"), "netdom (domain management)",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+(view|share|session|time|config)\b"), "recon")
| where isnotnull(action)
| eval cmd_masked=replace(replace(replace(CommandLine,"(?i)(user\s+\S+\s+)[^/\s]\S*","\1********"),"(?i)(/user:\S+\s+)
      \S+","\1********"),"(?i)(/pd?:)\S+","\1********")
| sort 0 _time
| table _time Computer User Image action cmd_masked
```

Line	What it does
Search lines	Program starts where the program is net.exe, net1.exe or netdom.exe.
eval action=case(...)	The sorting hat. Each match(...) line asks 'does the command fit this pattern?' and gives it a label if it does. The order matters: 'create user' is checked before 'possible password reset', so a /add command does not get the wrong label.
\bnet1?(\.exe)?\s+user	A word edge, then net or net1, optionally followed by .exe, then spaces, then the word user.

Line	What it does
\S+\s+\S+.*/add	A username, a space, a password, anything, then /add.
where isnotnull(action)	Throw away every command that got no label.
cmd_masked	Three nested replace calls. Each finds a password in a different spot (after user NAME, after /user:ACCOUNT, after /p: or /pd:) and swaps it for stars. The \1 means put back the part I kept, so the username stays and only the password is hidden. Passwords sit in clear text in command lines, so you do not want them on screen.

Example result (LAB, from your sysmon.json, using the same labelling rules):

_time	Computer	User	Image	action	cmd_masked
2024-06-06 09:31:57	DESKTOP-ND6FH5D	DESKTOP-ND6FH5D\LetsDefend	C:\Windows\System32\net.exe	create user	net user jumpadmin ******** /add
2024-06-06 09:31:57	DESKTOP-ND6FH5D	DESKTOP-ND6FH5D\LetsDefend	C:\Windows\System32\net1.exe	create user	C:\Windows\system32\net1 user jumpadmin ******** /add

Two rows for one command, net.exe then net1.exe a fraction of a second later. That is normal. The plain 'net user' command at 09:32:07 gets no label because it only lists users, so it is not shown.

The Active Directory tools

Active Directory (AD) is the school's master list of every student, teacher, computer and rule. The domain controller is the office where that list is kept. The tools below are the pens and erasers that can change the list.

Type	Program	What it does
Consoles (open inside mmc.exe)	dsa.msc	Users and Computers: create, delete and reset users and groups
	adsiedit.msc	Edits raw entries of the list, very powerful
	gpmc.msc	Group Policy: rules pushed to every computer
	dssite.msc, domain.msc, dnsmgmt.msc, schmmgmt.msc	Sites, trusts, DNS, and the schema (the rule-book of the list itself)
Program with windows	dsac.exe	Active Directory Administrative Center
	ldp.exe	A raw browser that can also edit
Command line	net user / net group with /domain	Add users, change groups
	dsadd, dsmod, dsrm, dsquery	Older command-line tools
	ldifde.exe, csvde.exe	Bulk import and export of the list
	dsacls.exe	Changes who may edit which entries
	ntdsutil.exe	Database tasks, including copying the whole list (very sensitive)
	setspn.exe	Changes service names, used in Kerberoasting setup
	repadmin.exe, dcdiag.exe, nltest.exe	Replication, health and trusts, mostly looking around
	netdom.exe	Joins or removes computers, resets machine passwords, builds trusts
PowerShell	New-ADUser, Set-ADAccountPassword, Set-ADUser, Add-ADGroupMember, Enable-ADAccount, Unlock-ADAccount, Set-ADAccountControl, Set-ADObject, Remove-ADObject, Set-ADComputer, New-GPO, Set-GPRegistryValue, New-GPLink	Create users, reset passwords, change groups and attributes, create and link policies. Commands starting Get-AD only look.
Attacker tools	Mimikatz, PowerView, Rubeus, SharpHound, AdFind, impacket	Steal passwords, change things, map the domain

You can catch these at three layers, because attackers can skip one but rarely all three.

Layer 1: the tool was launched.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| eval tool=case(
    match(Image,"(?i)\\\\mmc\.exe$") AND
          match(CommandLine,"(?i)(dsa|adsiedit|dssite|domain|gpmc|dnsmgmt|schmmgmt)\.msc"), "AD console via mmc",
    match(Image,"(?i)\\\\(dsac|ntdsutil|ldifde|csvde|dsacls|dsadd|dsmod|dsrm|ldp|setspn|repadmin|dcdiag|nltest|adfind)
          \.exe$"), "AD command-line tool",
    match(CommandLine,"(?i)\bnet1?(\.exe)?\s+(user|group)\s+.*/domain"), "net ... /domain")
| where isnotnull(tool)
| stats count, values(CommandLine) as commands, earliest(_time) as first, latest(_time) as last by Computer, User,
      tool
| convert ctime(first) ctime(last)
| sort - count
```

The first rule says: if the program is mmc.exe (the box every console opens in) and its command line names one of those .msc files, call it an AD console. The second rule matches the standalone tools by file name. The third matches net user or net group with /domain. Then it throws away everything without a label and tallies the rest. Example result (MADE-UP):

Computer	User	tool	count	commands
WS-0142	CORP\adm_bob	AD console via mmc	1	"mmc.exe" "C:\Windows\system32\dsa.msc"

Layer 2: the PowerShell commands. Commands typed into an open PowerShell window do not show in the process list, so you need PowerShell script block logging (event 4104) switched on. That event records the actual commands.

```
index=* sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational" EventCode=4104
| regex _raw="(?i)\b(New-ADUser|Set-ADAccountPassword|Set-ADUser|Add-ADGroupMember|Add-ADPrincipalGroupMembership|Remo
      ve-ADGroupMember|Enable-ADAccount|Disable-ADAccount|Unlock-ADAccount|Set-ADAccountControl|Set-ADObject|New-ADObj
      ect|Remove-ADObject|Move-ADObject|Set-ADComputer|New-ADComputer|Set-ADDefaultDomainPasswordPolicy|New-GPO|Set-GP
      RegistryValue|New-GPLink)\b"
| table _time host user UserID ScriptBlockText
| sort - _time
```

Keep only PowerShell events whose text contains one of those change-making commands as a whole word. Example result (MADE-UP):

_time	host	user	ScriptBlockText
2026-10-28 09:21:12	WS-0142	CORP\adm_bob	Add-ADGroupMember -Identity 'Domain Admins' -Members jumpadmin

Layer 3: the change itself, recorded on the domain controllers. This is the most reliable record because it does not care which tool was used.

```
index=* sourcetype=*WinEventLog:Security (EventCode=4720 OR EventCode=4722 OR EventCode=4723 OR EventCode=4724 OR
      EventCode=4725 OR EventCode=4726 OR EventCode=4728 OR EventCode=4729 OR EventCode=4732 OR EventCode=4733 OR
      EventCode=4738 OR EventCode=4739 OR EventCode=4742 OR EventCode=4756 OR EventCode=4757 OR EventCode=4767 OR
      EventCode=5136 OR EventCode=5137 OR EventCode=5141)
| eval action=case(EventCode=4720,"user created", EventCode=4722,"user enabled", EventCode=4723,"password changed by
      owner", EventCode=4724,"password reset by someone else", EventCode=4725,"user disabled", EventCode=4726,"user
      deleted", EventCode=4728,"added to global group", EventCode=4729,"removed from global group",
      EventCode=4732,"added to local group", EventCode=4733,"removed from local group", EventCode=4738,"user account
      changed", EventCode=4739,"domain policy changed", EventCode=4742,"computer account changed",
      EventCode=4756,"added to universal group", EventCode=4757,"removed from universal group",
      EventCode=4767,"account unlocked", EventCode=5136,"AD object modified", EventCode=5137,"AD object created",
      EventCode=5141,"AD object deleted")
| eval who=coalesce(SubjectUserName, Subject_Account_Name), target=coalesce(TargetUserName, Target_Account_Name,
      ObjectDN)
| stats count, values(action) as actions, dc(target) as objects_changed, earliest(_time) as first, latest(_time) as
      last by who, host
| convert ctime(first) ctime(last)
| sort - count
```

The long OR list is the set of AD change events. The case turns the number into words so you read 'user created' instead of 4720. who is the person who made the change, target is the thing changed. The stats line gives, per person and computer, how many changes, what kinds, how many DIFFERENT things were touched, and when. Events 5136, 5137 and 5141 only exist if 'Audit Directory Service Changes' is switched on. By default the controllers log successful changes well but failed attempts only if failure auditing is on, so to catch attempts you mostly rely on layers 1 and 2. Example result (MADE-UP):

who	host	count	actions	objects_changed
adm_bob	DC01	2	user created, added to global group	2

The most useful upgrade is a list of your real AD admins saved as a lookup file. These two lines check each person against the list and keep only those who are NOT on it:

```
| lookup ad_admins.csv user as who OUTPUT is_admin
| where isnull(is_admin)
```

netadmin.exe: not an AD tool

There is no built-in Windows program called netadmin. netadmin.exe is the main program of DEKSI Network Administrator, a third-party tool that scans a network, finds computers, draws them on a map and monitors them. Other unrelated products also use the same filename, such as an old Novell NetWare tool, so the name alone proves nothing. It cannot change AD. If it appears on a machine where nobody installed it, treat it as network scanning, or as malware using a trusted-looking name.

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 (Image="*\\netadmin.exe" OR
      Image="*\\netadm.exe")
| eval expected_path=if(match(Image,"(?i)^C:\\\\Program Files( \(x86\))?\\\\(DEKSI Network
      Administrator|netadm)\\\\"),"yes","NO")
| stats count, earliest(_time) as first, latest(_time) as last, values(Hashes) as hashes, values(ParentImage) as
      parents, values(User) as users by Computer, Image, expected_path
| convert ctime(first) ctime(last)
```

Every time something called netadmin.exe started, with a flag for whether it sits in the usual install folder. The installer lets users choose another folder, so NO is a flag, not proof. Different hashes on different machines, or a run from Downloads, Temp or Public, is the strongest sign of an imitation. Example result (MADE-UP):

Computer	Image	expected_path	count	users
WS-0142	C:\Users\bob.smith\Downloads\netadmin.exe	NO	1	CORP\bob.smith

```
index=* sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 Image="*\\netadmin.exe"
      earliest=-24h
| bin _time span=10m
| stats dc(DestinationIp) as hosts_contacted, values(DestinationPort) as ports, count by _time, Computer, User
| where hosts_contacted>=20
```

Connections made by netadmin.exe grouped into 10 minute buckets. It counts how many DIFFERENT computers it contacted in each bucket and keeps buckets of 20 or more, which looks like scanning. Tune the 20 to your network.

7. How to decide

It is probably fine if the program is something like consent.exe, mstsc.exe or a scheduled task, step B shows it is regular, and the user or system owner confirms it. Escalate if the program is odd or runs from a user folder, the credentials belong to an admin the person should not have, the target is a new server, the card appears on many machines, or something suspicious follows in step C. Always ask the user and the machine's owner whether they did admin work that day. In an incident, also contact the user's manager, check whether the used account was reset or signed in elsewhere, and isolate the host if a suspicious program or outside connection shows up.

8. Honest limits

None of these queries have run on your Splunk. Open one real event and check the field names first (for example Account_Name versus TargetUserName). A query that returns nothing may mean the logging is switched off, not that nothing happened. The MADE-UP examples show the shape of a result, not real data. The LAB example comes from your sysmon.json using the same labelling rules, checked in Python, not Splunk.